

plots

February 18, 2026

1 Python Libraries for Data Visualization

Recap: Each visualization serves a specific purpose; choose the one that best fits your data.

Best Practices: - Always include labels and titles. - Use color effectively. - Consider interactive plots for reports or dashboards. - Use combination charts when necessary (e.g., bar and line together).

Matplotlib: The foundational plotting library, used for basic charts.

Seaborn: Built on top of Matplotlib, ideal for statistical plots.

Plotly: Used for interactive and web-based visualizations.

1.1 1. Line chart:

Use Case: Showing trends over time.

Library: Matplotlib.

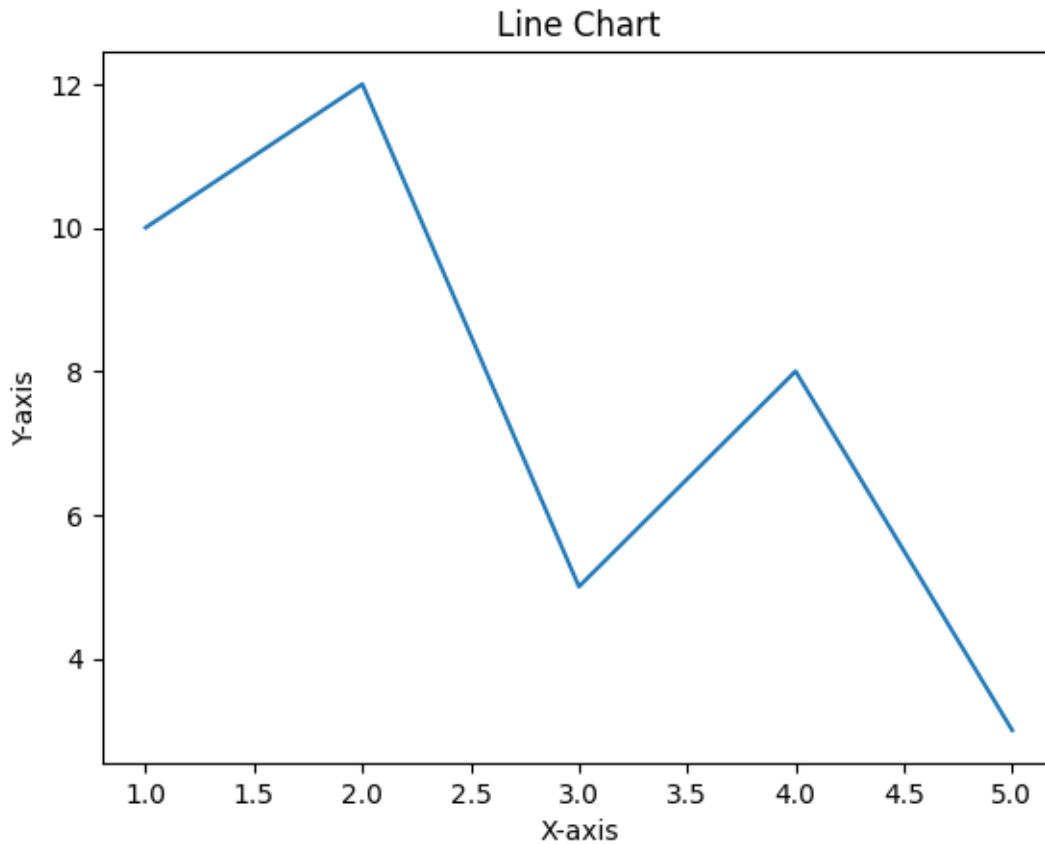
```
[3]: from IPython import display
display.Image(filename="images/image.png", width=400)
```

[3]:



```
[1]: import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 12, 5, 8, 3]
```

```
plt.plot(x, y)
plt.title('Line Chart')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```



1.1.1 Customizing plots

You can customize the appearance of your plots using various options. For example, you can change the line color, line width, marker style, and more:

```
[ ]: import matplotlib.pyplot as plt

# Create some data
x = ['2021', 2022, 2023, 2024, 2025]
ESS = [71, 54, 42, 48, 41]
MCA = [54, 51, 47, 65, 58]

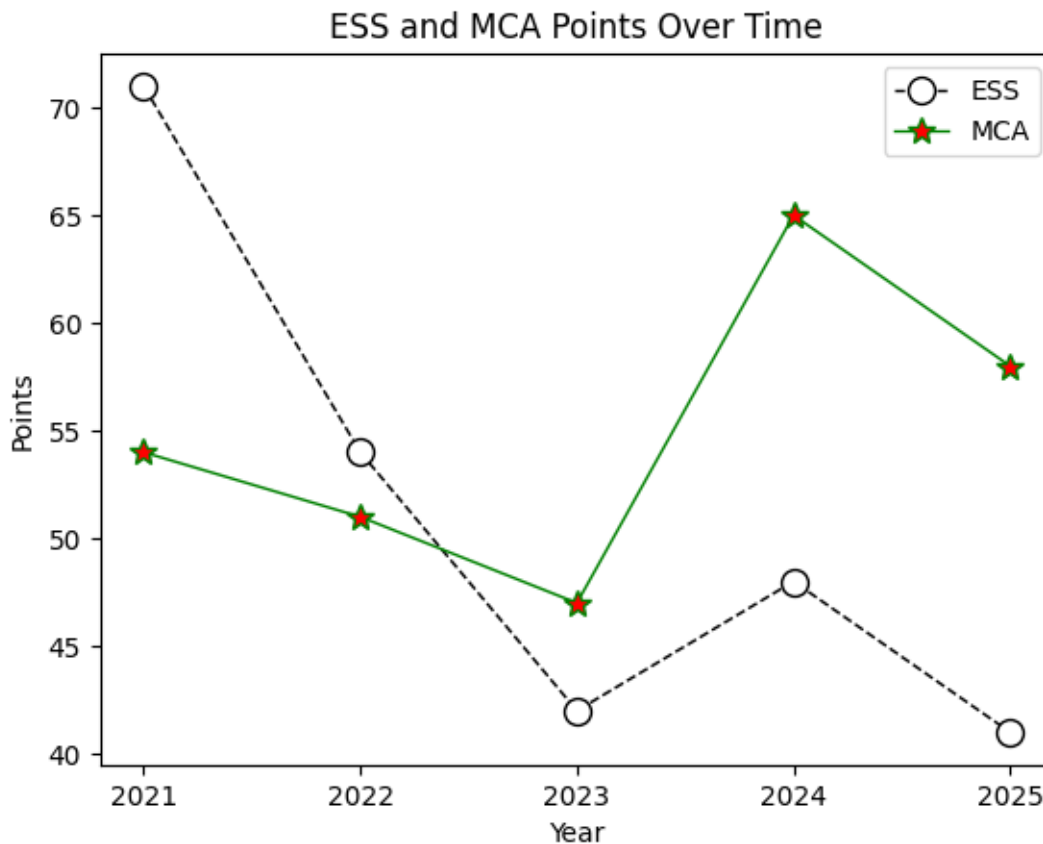
# Create a plot with custom options
```

```

plt.plot(x, ESS, color='black', linewidth=1, marker='o', markersize=10,
        ↪markerfacecolor='white', linestyle='--')
plt.plot(x, MCA, color='green', linewidth=1, marker='*', markersize=10,
        ↪markerfacecolor='red')
plt.title('ESS and MCA Points Over Time')
plt.xlabel('Year')
plt.ylabel('Points')
plt.legend(['ESS', 'MCA'])

# Show the plot
plt.show()

```



```

[3]: import yfinance as yf
import matplotlib.pyplot as plt
# Download historical data for Apple Inc. (AAPL)
data = yf.download('AAPL', start='2023-01-01', end='2025-12-31')
# Plot the closing prices
plt.figure(figsize=(10, 5))
plt.plot(data.index, data['Close'], label='AAPL Closing Prices')

```

```
plt.title('Apple Inc. (AAPL) Closing Prices in 2023-2025')
plt.xlabel('Date')
plt.ylabel('Price (USD)')
plt.legend()
plt.grid()
plt.show()
```

[*****100%*****] 1 of 1 completed

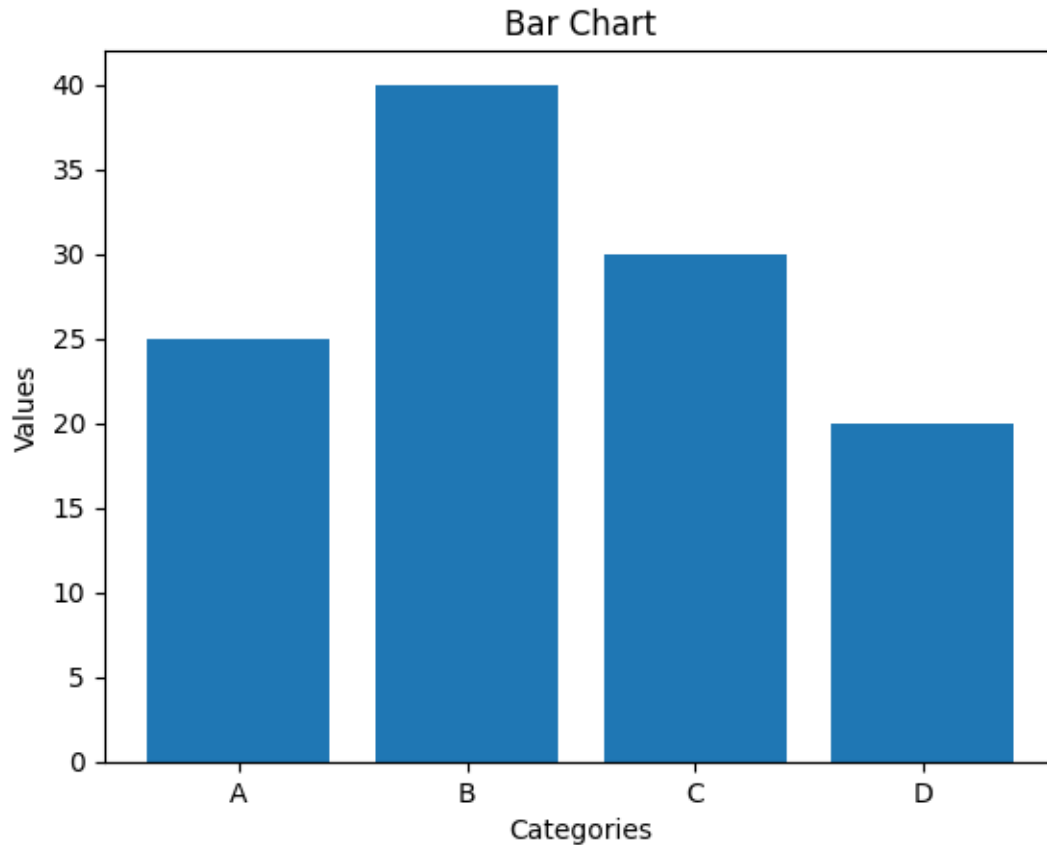


1.2 2. Bar chart:

Use Case: Comparing different categories or discrete items..

Library: Matplotlib.

```
[4]: import matplotlib.pyplot as plt
categories = ['A', 'B', 'C', 'D']
values = [25, 40, 30, 20]
plt.bar(categories, values)
plt.title('Bar Chart')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.show()
```



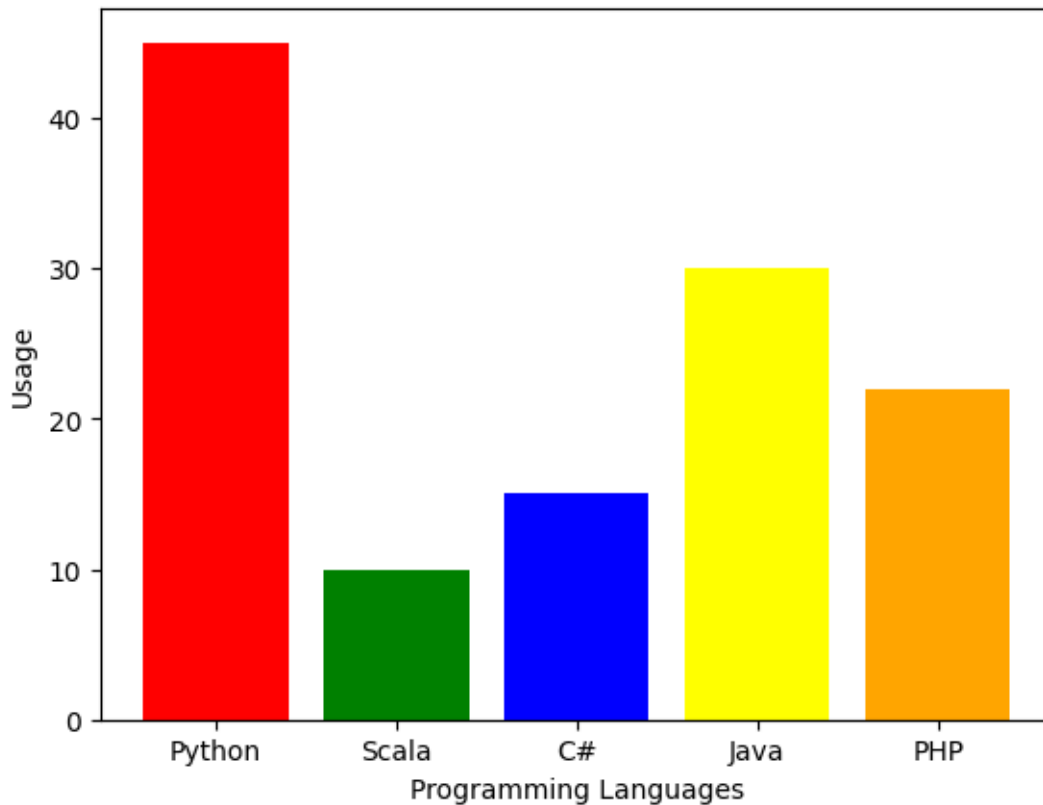
1.3 Coloured Bar Graph:

```
[5]: import matplotlib.pyplot as pyplot # Set up the data
labels = ('Python', 'Scala', 'C#', 'Java', 'PHP')
index = (1, 2, 3, 4, 5) # provides Locations on x axis
sizes = [45, 10, 15, 30, 22]

# Set up the bar chart
pyplot.bar(index, sizes, tick_label=labels, color=('red','green', 'blue', 'yellow', 'orange'))

# Configure the Layout
pyplot.ylabel("Usage")
pyplot.xlabel("Programming Languages")

# Display the chart
pyplot.show()
```



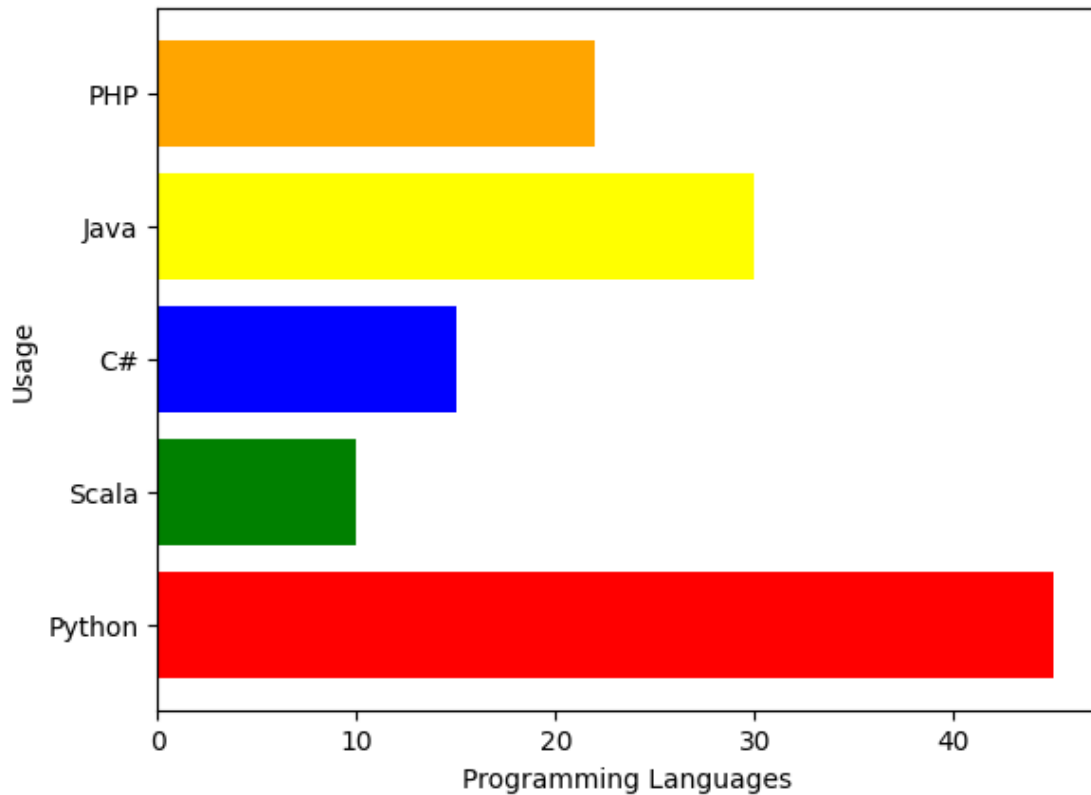
1.4 Horizontal Bar Graph:

```
[6]: import matplotlib.pyplot as pyplot # Set up the data
labels = ('Python', 'Scala', 'C#', 'Java', 'PHP')
index = (1, 2, 3, 4, 5) # provides Locations on x axis
sizes = [45, 10, 15, 30, 22]

# Set up the bar chart
pyplot.barh(index, sizes, tick_label=labels, color=('red', 'green', 'blue', 'yellow', 'orange'))

# Configure the Layout
pyplot.ylabel("Usage")
pyplot.xlabel("Programming Languages")

# Display the chart
pyplot.show()
```



1.5 Stacked Bar Graph:

```
[7]: import matplotlib.pyplot as plt

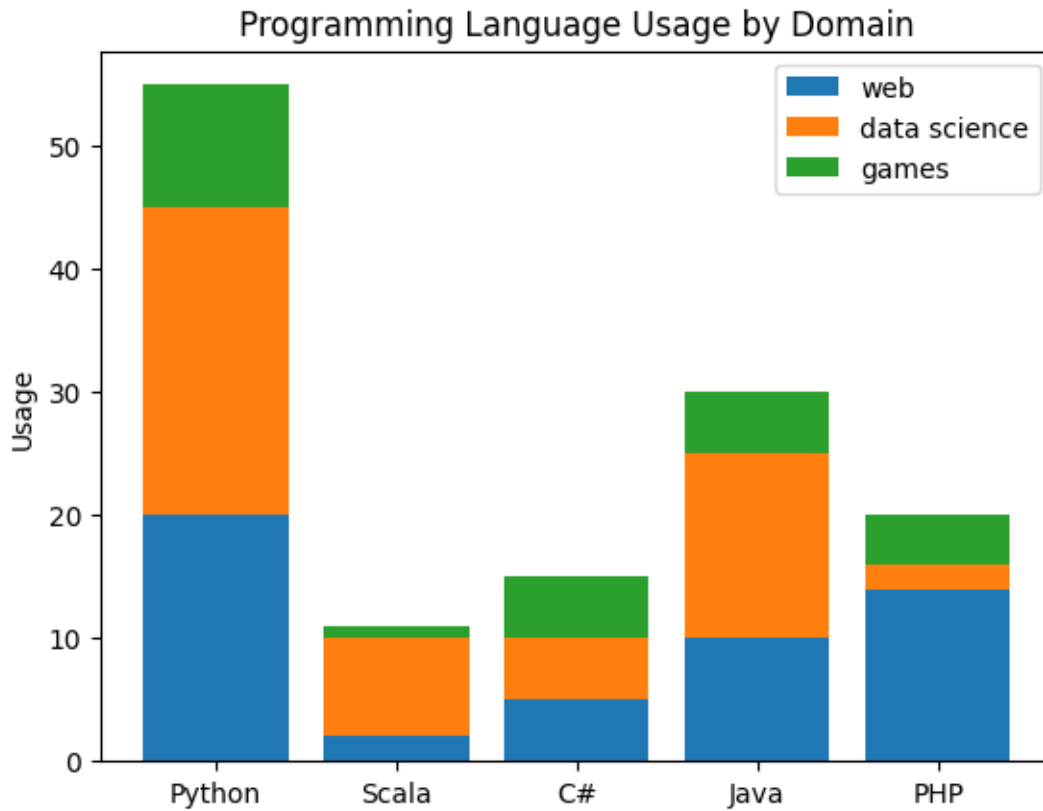
# Set up the data
labels=['Python', 'Scala', 'C#', 'Java', 'PHP']
index = [1, 2, 3, 4, 5] # provides Locations on x axis
web_usage = [20, 2, 5, 10, 14]
data_science_usage = [25, 8, 5, 15, 2]
games_usage = [10, 1, 5, 5, 4]

# Set up the bar chart
plt.bar(index, web_usage, tick_label=labels, label='web')
plt.bar(index, data_science_usage, tick_label=labels, label='data science',
        bottom=web_usage)
web_and_games_usage = [web_usage[i] + data_science_usage[i] for i in range(0,
        len(web_usage))]
plt.bar(index, games_usage, tick_label=labels, label='games',
        bottom=web_and_games_usage)

# Configure the Layout
```

```
plt.ylabel('Usage')
plt.title('Programming Language Usage by Domain')
plt.legend()

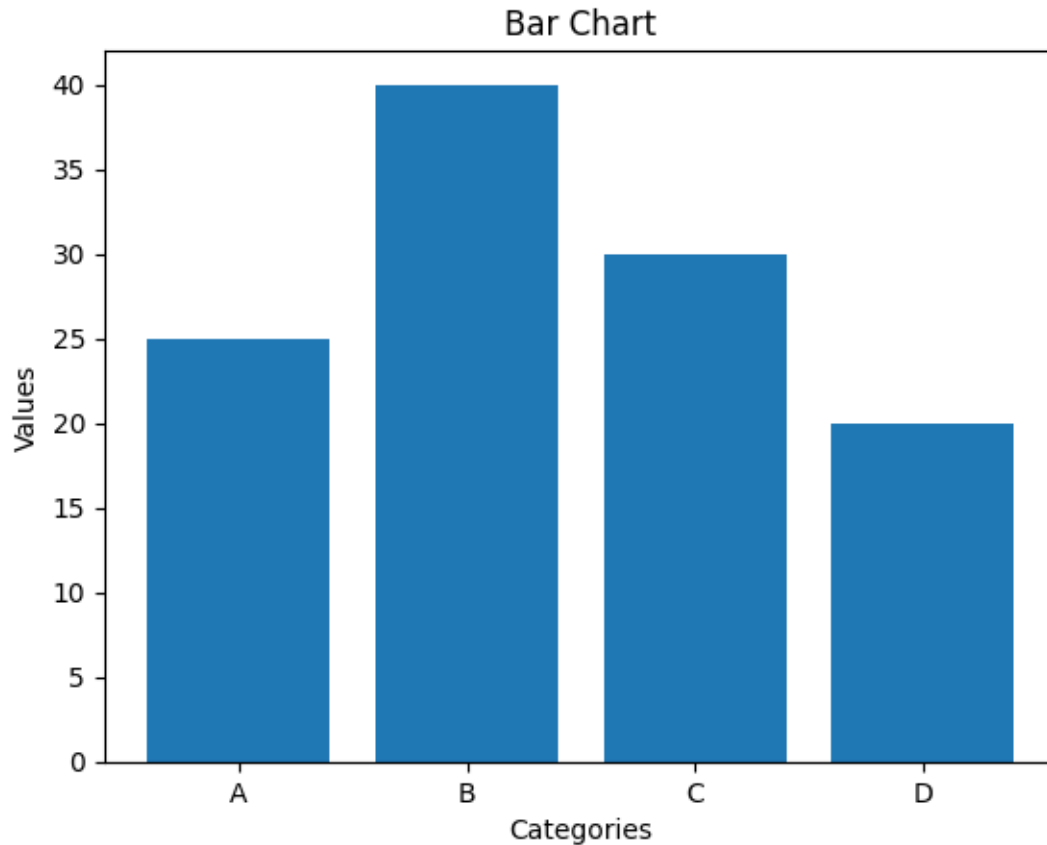
# Display the chart
plt.show()
```



1.6 Bar graph plot using different Python libraries

1.6.1 1. Matplotlib:

```
[8]: # Bar graph plot using different Python libraries
# 1. Matplotlib:
import matplotlib.pyplot as plt
categories = ['A', 'B', 'C', 'D']
values = [25, 40, 30, 20]
plt.bar(categories, values)
plt.title('Bar Chart')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.show()
```



1.6.2 2. plotly:

```
[38]: # Bar graph plot using different Python libraries
# 2. Plotly
import plotly.express as px
categories = ['A','B', 'C', 'D']
values = [25, 40, 30, 20]

# Create an interactive bar chart
fig = px.bar(x=categories, y=values, labels={'x':'Categories', 'y':'Values'},
             title='Bar Chart example')
fig.show()
```

1.6.3 3. Seaborn:

```
[10]: # Bar graph plot using different Python libraries
# 3. Seaborn
import seaborn as sns
import matplotlib.pyplot as plt
# Sample data
```

```

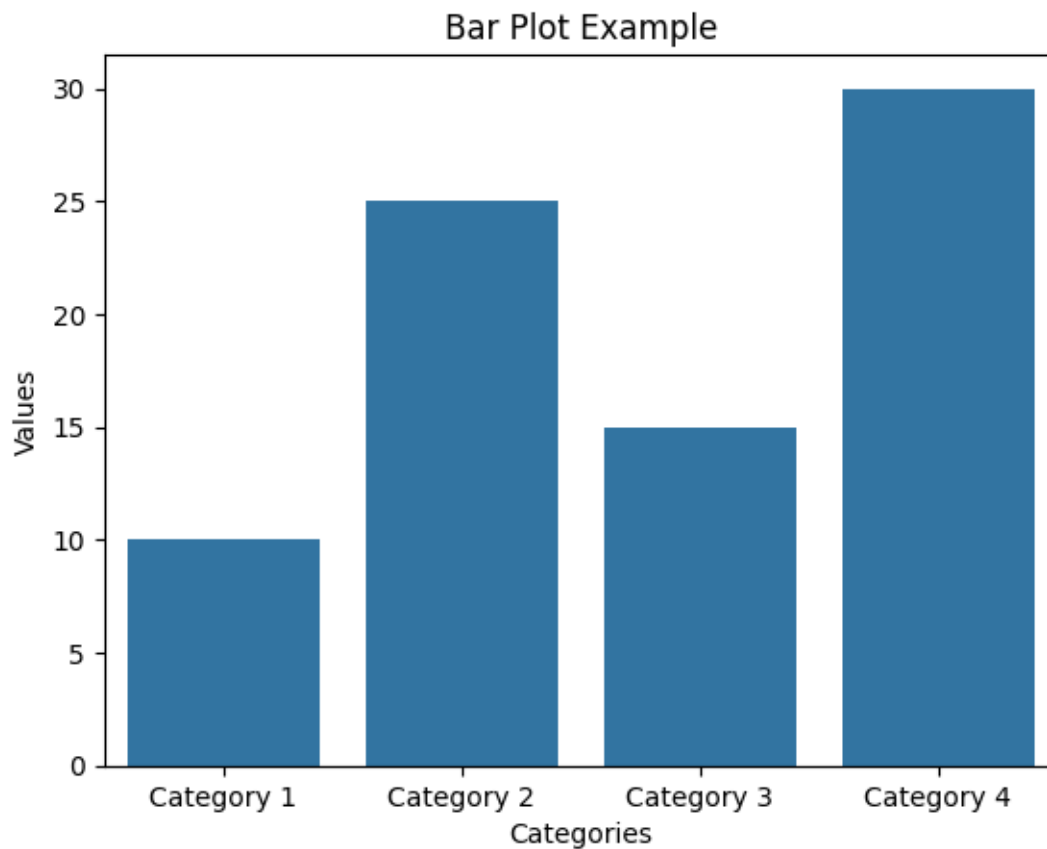
categories = ['Category 1', 'Category 2', 'Category 3', 'Category 4']
values = [10, 25, 15, 30]

# Create a bar plot using Seaborn
sns.barplot(x=categories, y=values)

# Adding Labels and title
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Bar Plot Example')

# Show the plot
plt.show()

```



1.6.4 4. Bokeh

```

[11]: # Bar graph plot using different Python libraries
# 4. Bokeh
from bokeh.plotting import figure, show
from bokeh.io import output_notebook

```

```

# sample data
categories = ['A', 'B', 'C', 'D']
values = [10, 25, 30, 15]

# Create a bar chart using Bokeh
p=figure(x_range=categories, title="Bar Chart Example",
        x_axis_label='Categories', y_axis_label='Values')
p.vbar(x=categories, top=values, width=0.5)

# Display the chart
output_notebook()
show(p)

```

```

[12]: # Bar graph plot using different Python libraries
# 5. Altair
import altair as alt
import pandas as pd

# Sample data
data = pd.DataFrame({
    "categories": ['Category 1', 'Category 2', 'Category 3', 'Category 4'],
    "values": [10, 25, 15, 30]
})

# Create a bar chart using Altair
chart = alt.Chart(data).mark_bar().encode(
    x='categories',
    y='values'
).properties(
    title='Bar Chart Example'
)
chart.show()

```

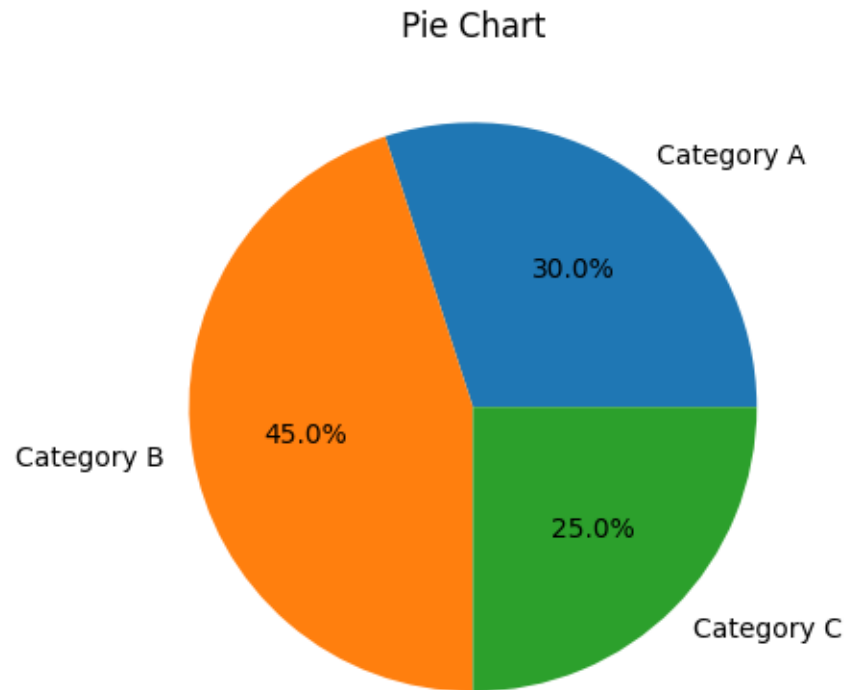
alt.Chart(...)

1.7 3. Pie chart:

```

[13]: import matplotlib.pyplot as plt
labels = ['Category A', 'Category B', 'Category C']
sizes = [30, 45, 25]
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title('Pie Chart')
plt.show()

```



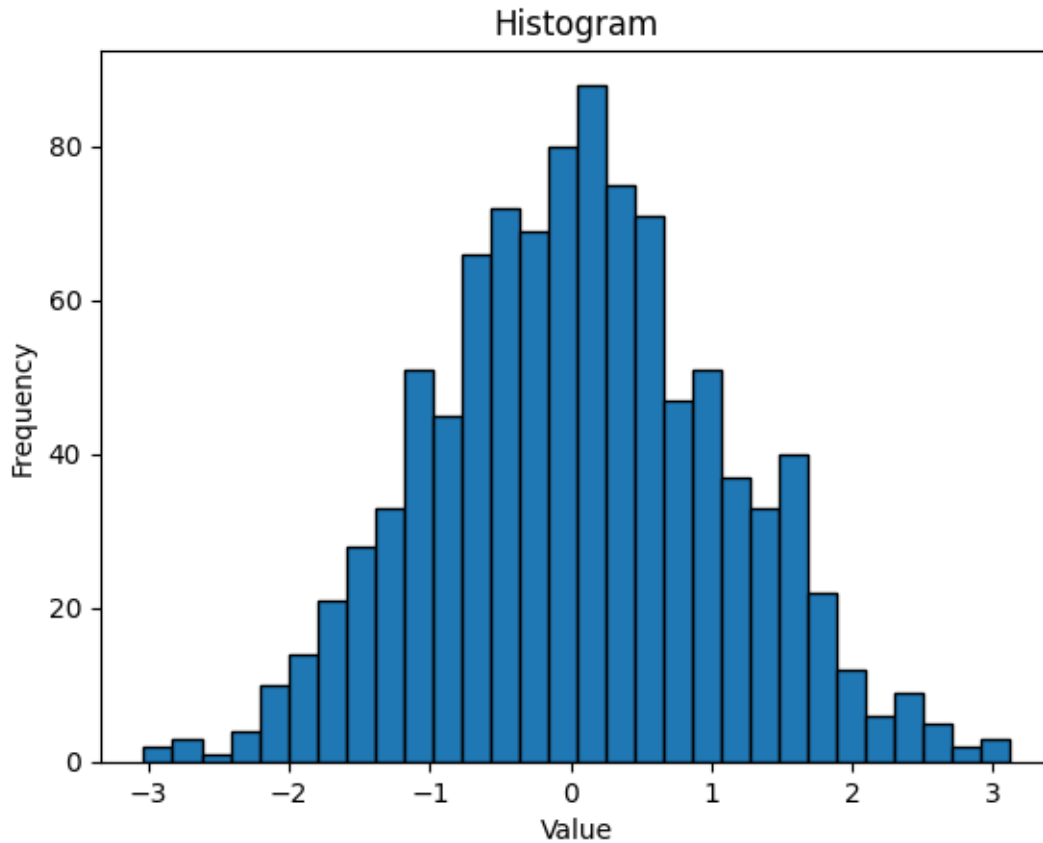
1.8 4. Histogram:

Use Case: Showing the distribution of data.

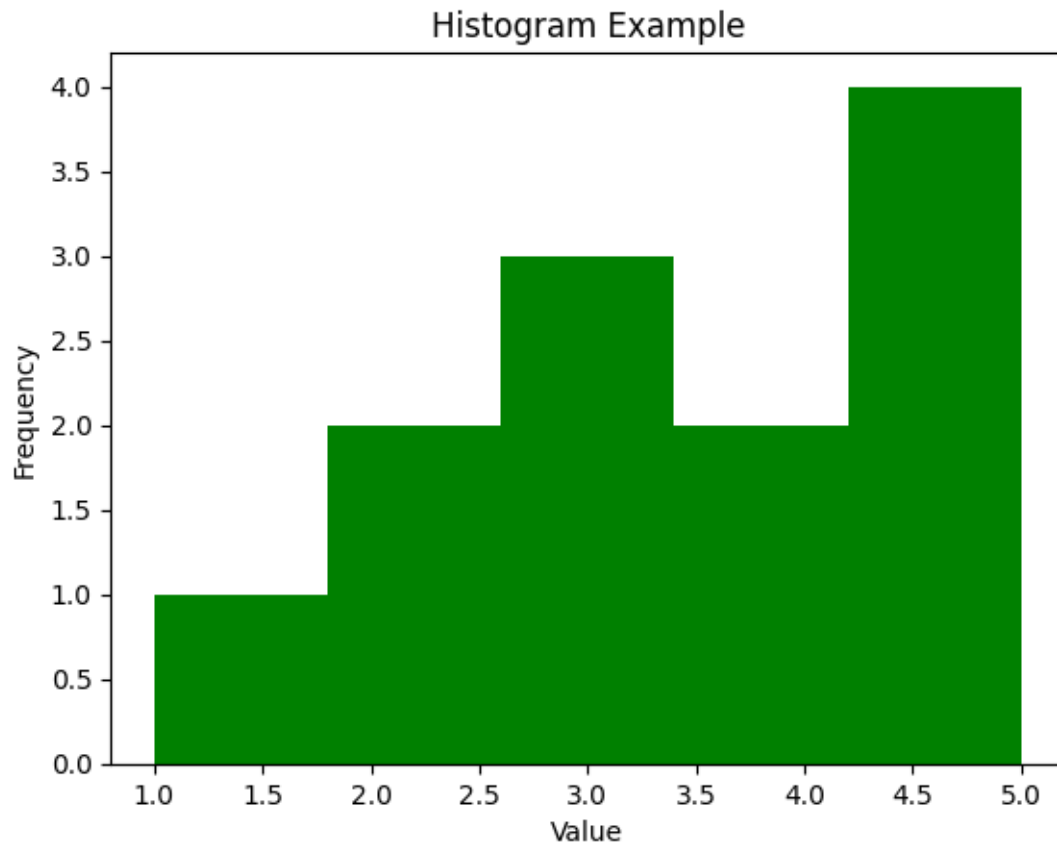
Library: Matplotlib.

```
[14]: import numpy as np
import matplotlib.pyplot as plt

data = np.random.randn(1000)
plt.hist(data, bins=30, edgecolor='black')
plt.title('Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```



```
[15]: import matplotlib.pyplot as plt # Import the Matplotlib Library
data = [1, 2, 2, 3, 3, 3, 4, 4, 5, 5, 5, 5] # Sample data to plot
# Create Histogram
plt.hist(data, bins=5, color='green') # Create a histogram with 5 bins and
    ↪ green bars
plt.title('Histogram Example') # Add a title to the plot
plt.xlabel('Value') # Label for the X-axis
plt.ylabel('Frequency') # Label for the Y-axis (frequency of each bin)
plt.show() # Display the plot
```



1.9 Grouped Bar Graph

```
[16]: import matplotlib.pyplot as pyplot
BAR_WIDTH = 0.35
# set up grouped bar charts
teama_results = (66, 75, 56, 62, 58)
teamb_results = (55, 68, 80, 73, 55)

# Set up the index for each bar
index_teama = (1, 2, 3, 4, 5)
index_teamb = [i + BAR_WIDTH for i in index_teama]

# Determine the mid point for the ticks
ticks = [i + BAR_WIDTH / 2 for i in index_teama]
tick_labels = ('Lab 1', 'Lab 2', 'Lab 3', 'Lab 4', 'Lab 5')

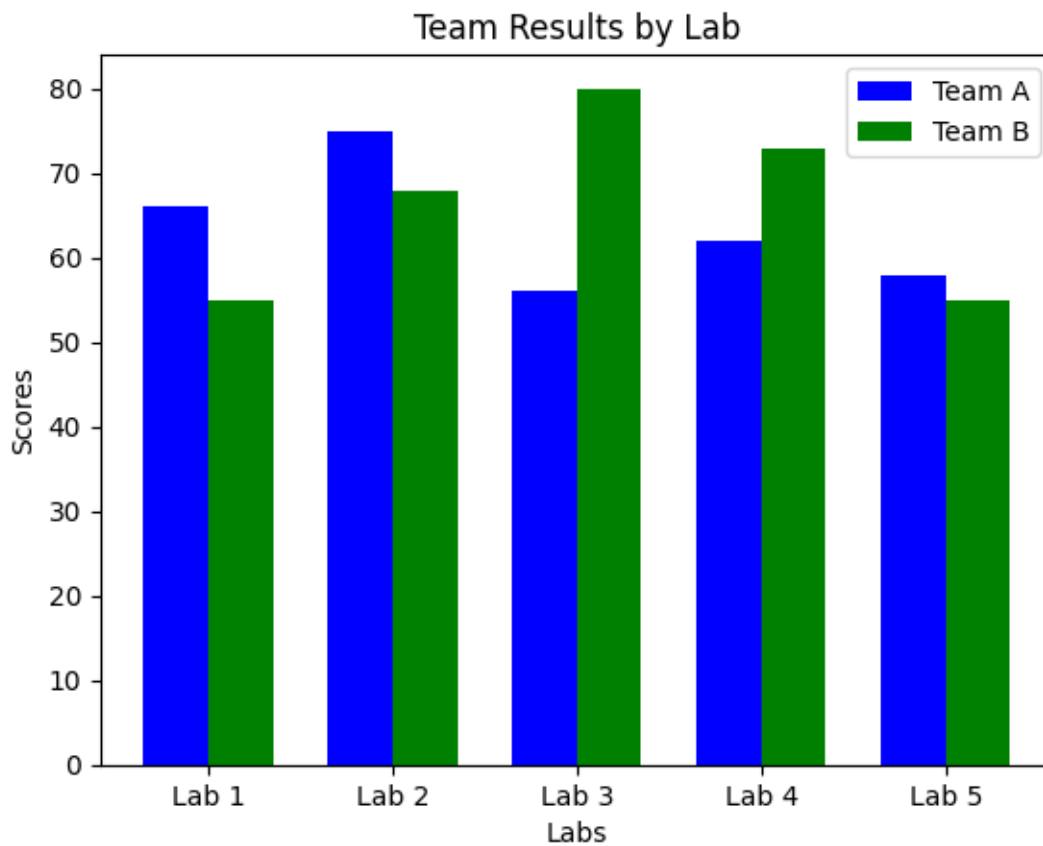
# Plot the bar charts
pyplot.bar(index_teama, teama_results, BAR_WIDTH, color="b", label='Team A')
pyplot.bar(index_teamb, teamb_results, BAR_WIDTH, color="g", label='Team B')
```

```

# Configure the layout
pyplot.xlabel('Labs')
pyplot.ylabel('Scores')
pyplot.title('Team Results by Lab')
pyplot.xticks(ticks, tick_labels)
pyplot.legend()

# Display the chart
pyplot.show()

```



1.10 5. Scatter plot:

Use Case: Showing relationships between two variables.

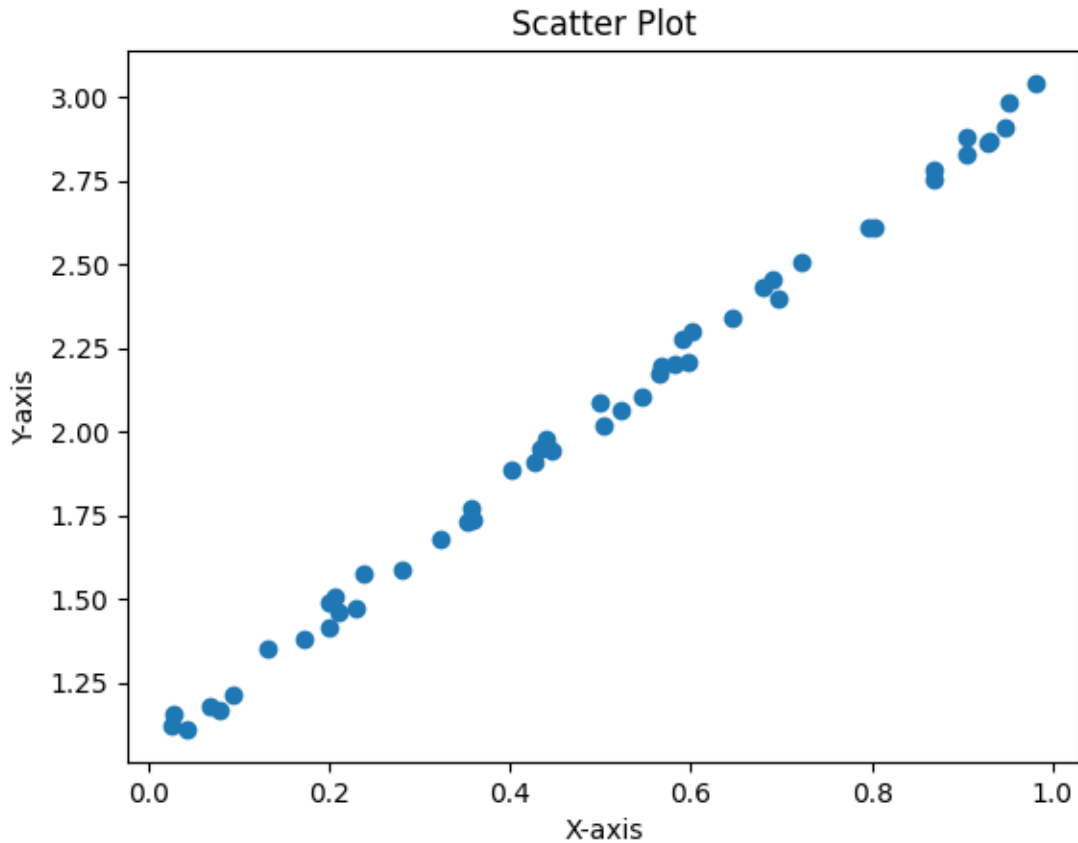
Library: Matplotlib.

```

[17]: import numpy as np
import matplotlib.pyplot as plt
x = np.random.rand(50)
y = 2*x + 1 + 0.1 * np.random.rand(50)

```

```
plt.scatter(x, y)
plt.title('Scatter Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```



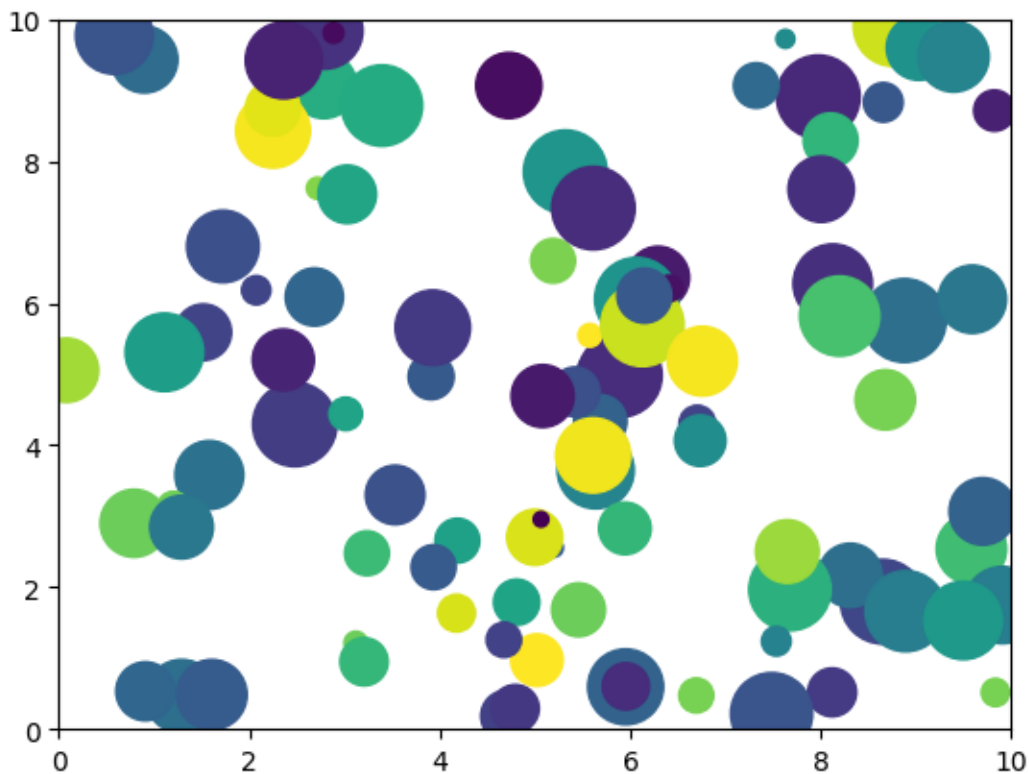
1.11 Animated scatter plot using Python

```
[37]: import matplotlib.pyplot as plt
import matplotlib.animation as animation
import numpy as np
num_points = 100
x, y = np.random.rand(2, num_points) * 10
colors = np.random.rand(num_points)
sizes = np.random.rand(num_points) * 1000
fig, ax = plt.subplots()
scat = ax.scatter(x, y, c=colors, s=sizes)
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)
```

```
def animate(i):
    scat.set_offsets(np.c_[x + np.random.randn(num_points) * 0.1, y + np.random.
↪randn(num_points) * 0.1])
    return scat,
ani = animation.FuncAnimation(fig, animate, frames=100, interval=50)
plt.show()
```

d:\Private\GitHub\pythonCourse\2025-2026\.venv\Lib\site-packages\matplotlib\animation.py:908: UserWarning:

Animation was deleted without rendering anything. This is most likely not intended. To prevent deletion, assign the Animation to a variable, e.g. `anim`, that exists until you output the Animation using `plt.show()` or `anim.save()`.



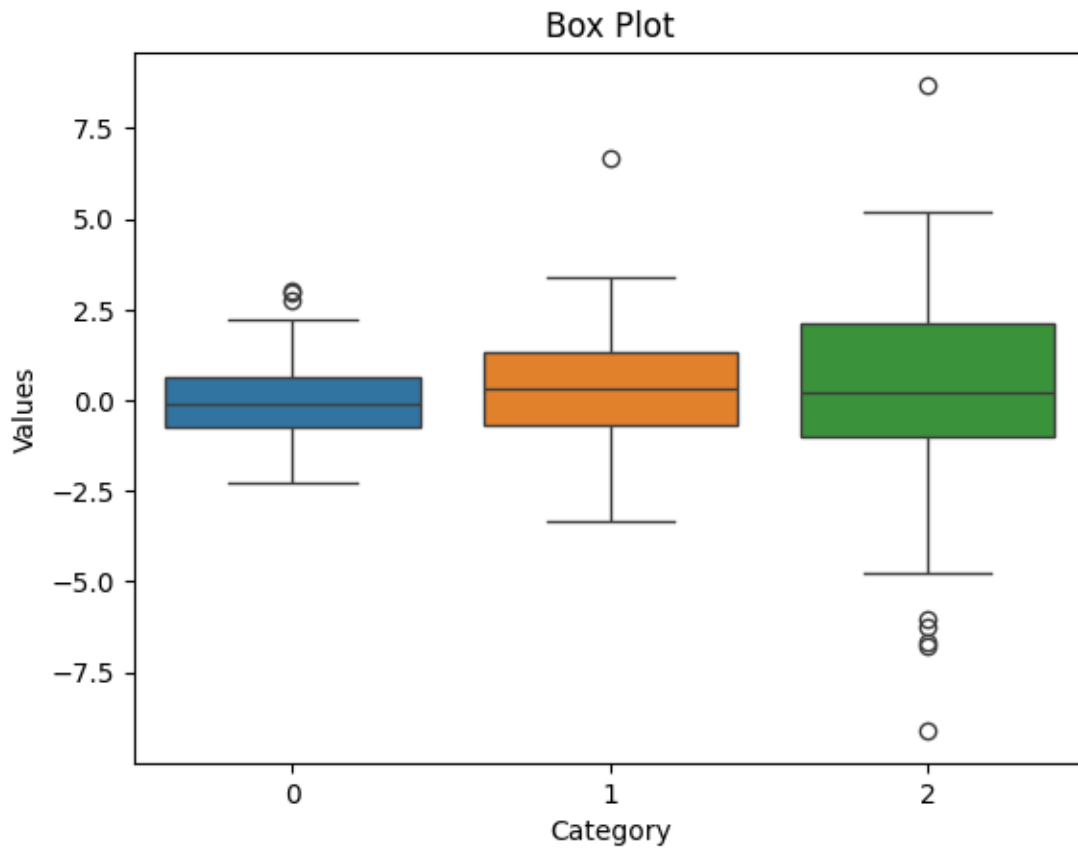
1.12 6. Box and whisker plots in Python:

Use Case: Summarizing the distribution of a dataset, showing quartiles and outliers.

Library: Seaborn

```
[19]: import seaborn as sns
import numpy as np
data = [np.random.normal(0, std, 100) for std in range(1, 4)]

sns.boxplot(data=data)
plt.title('Box Plot')
plt.xlabel('Category')
plt.ylabel('Values')
plt.show()
```



```
[20]: import matplotlib.pyplot as plt

data = [25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75]

# create a box and whisker plot with custom data

box_color = 'dodgerblue'
median_color = 'red'
whisker_color = 'green'
```

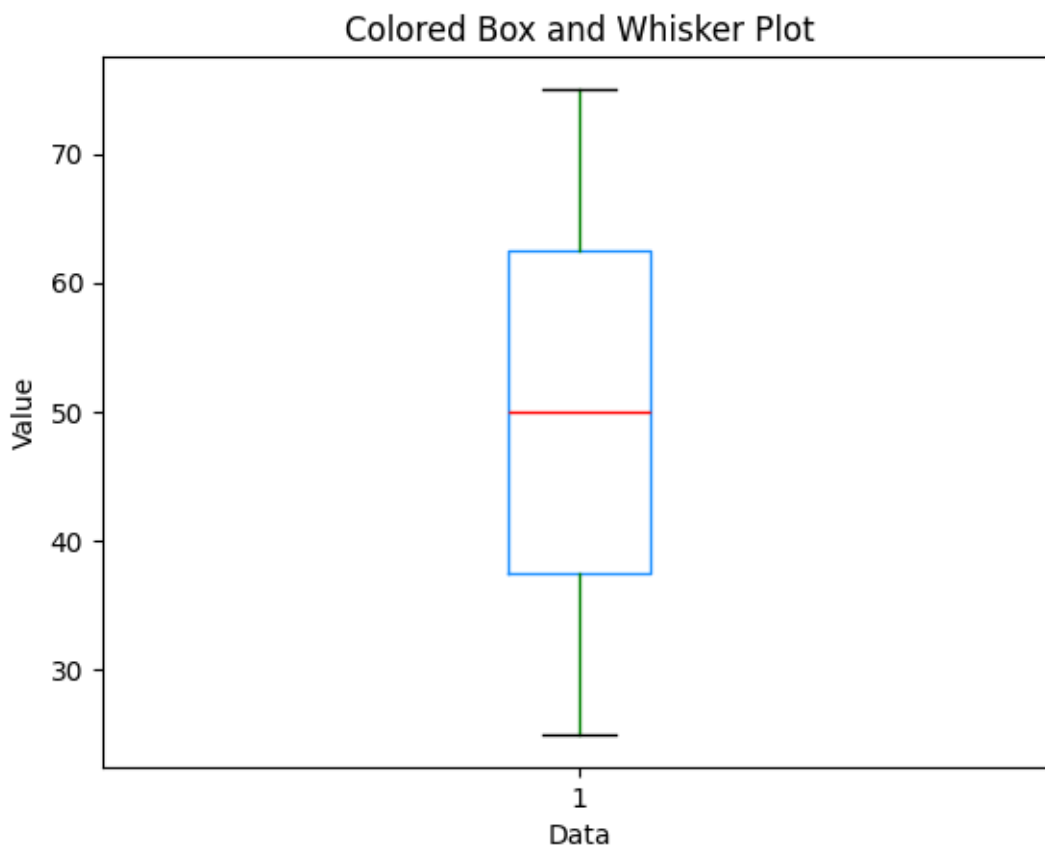
```

flier_color = 'purple'

plt.boxplot(data, boxprops={'color': box_color}, medianprops={'color': □
    ↳median_color}, whiskerprops={'color': whisker_color}, □
    ↳flierprops={'markerfacecolor': flier_color})

# Add Labels and a title plt.xlabel('Data')
plt.xlabel('Data')
plt.ylabel('Value')
plt.title('Colored Box and Whisker Plot')
# Display the plot
plt.show()

```



1.13 7. Violin plot:

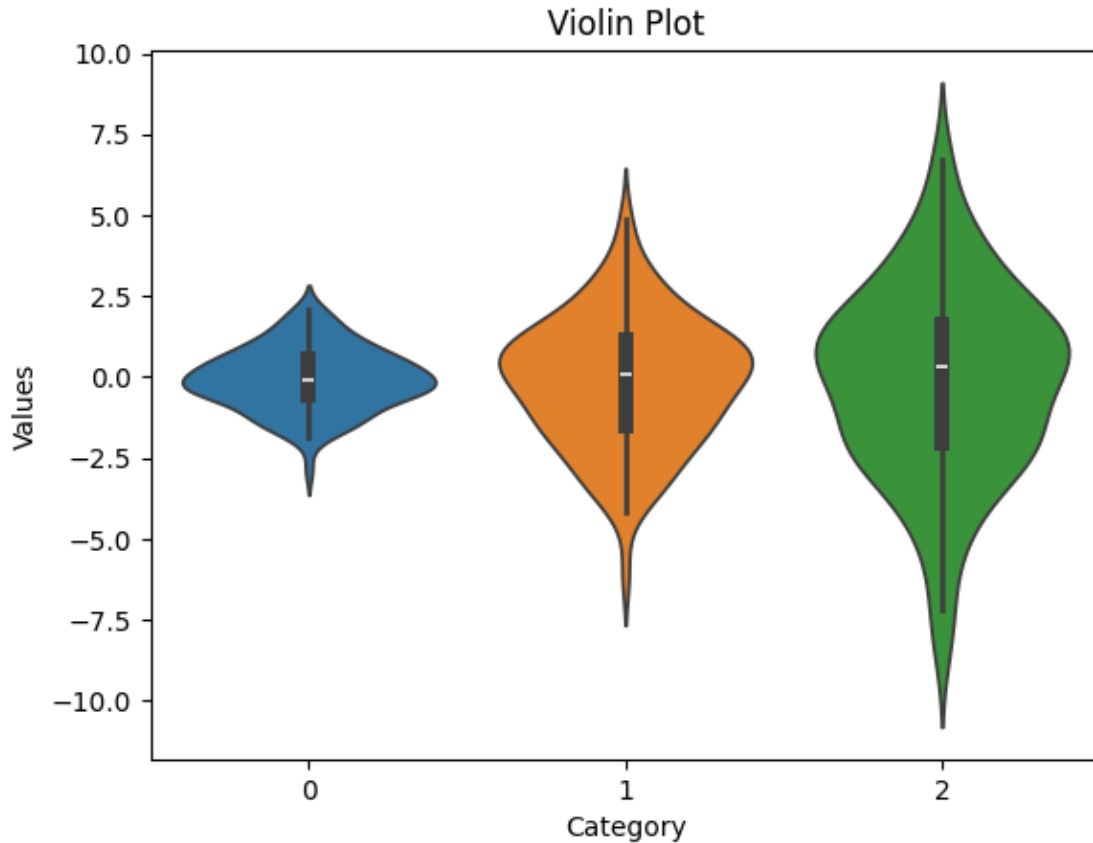
```

[21]: import seaborn as sns
import numpy as np

data = [np.random.normal(0, std, 100) for std in range(1, 4)]

```

```
sns.violinplot(data=data)
plt.title('Violin Plot')
plt.xlabel('Category')
plt.ylabel('Values')
plt.show()
```



1.14 8. Heatmap:

Use Case: Visualizing data in matrix form (correlation, confusion matrix).

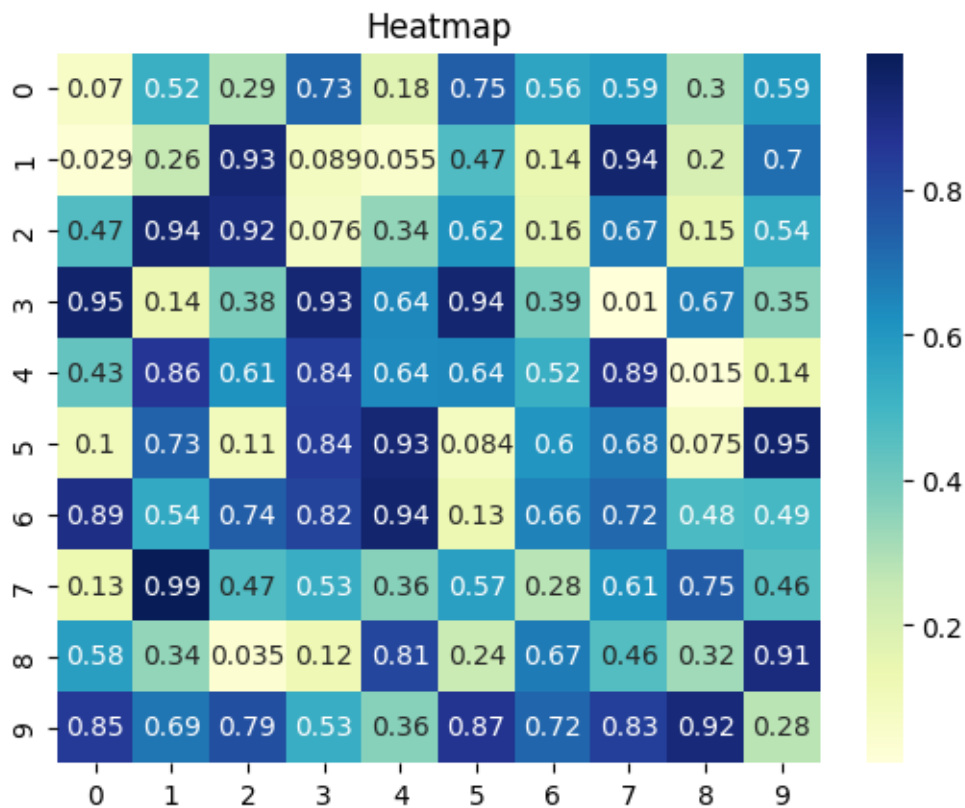
Library: Seaborn.

```
[22]: import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

data = np.random.rand(10, 10)

# sns.heatmap(data, annot=True) # Annotate cells with data values
sns.heatmap(data, annot=True, cmap='YlGnBu') # Use a colormap
```

```
# sns.heatmap(data, annot=True, cmap='plasma') # Use a colormap
plt.title('Heatmap')
plt.show()
```

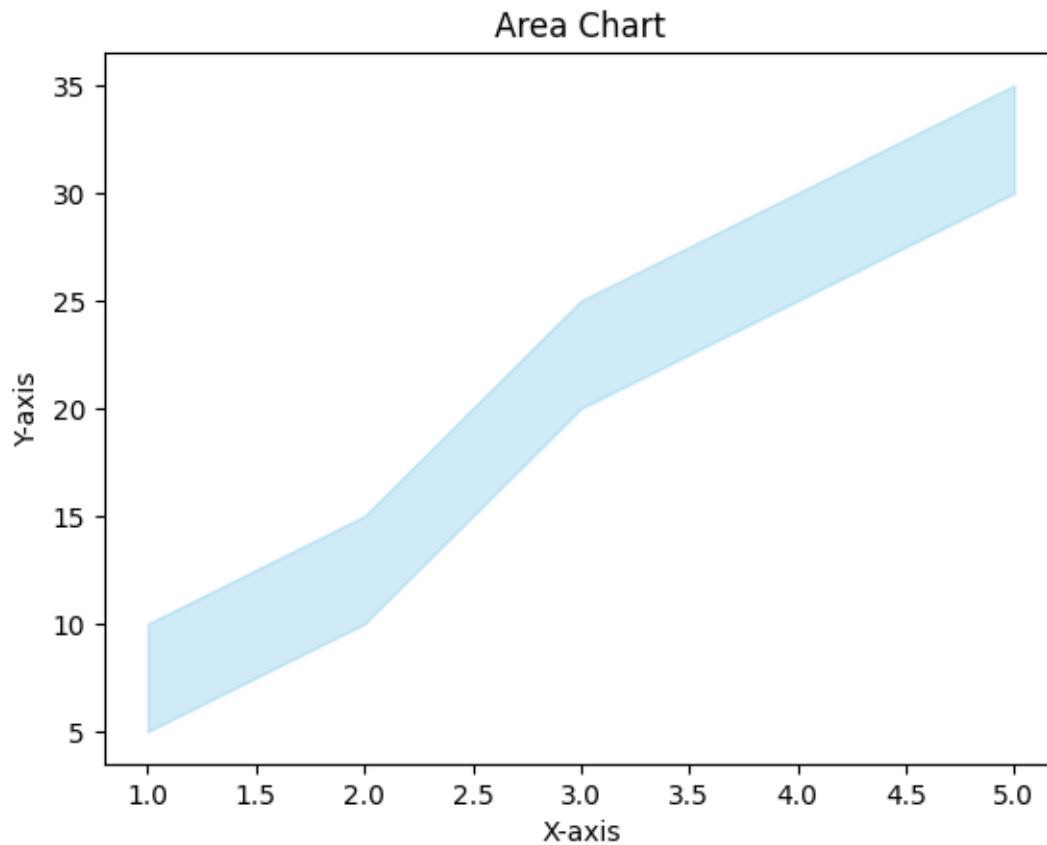


1.15 9. Area Chart:

```
[23]: import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y1 = [10, 15, 25, 30, 35]
y2 = [5, 10, 20, 25, 30]

plt.fill_between(x, y1, y2, color='skyblue', alpha=0.4) # alpha is the
↳ transparency level
plt.title('Area Chart')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```

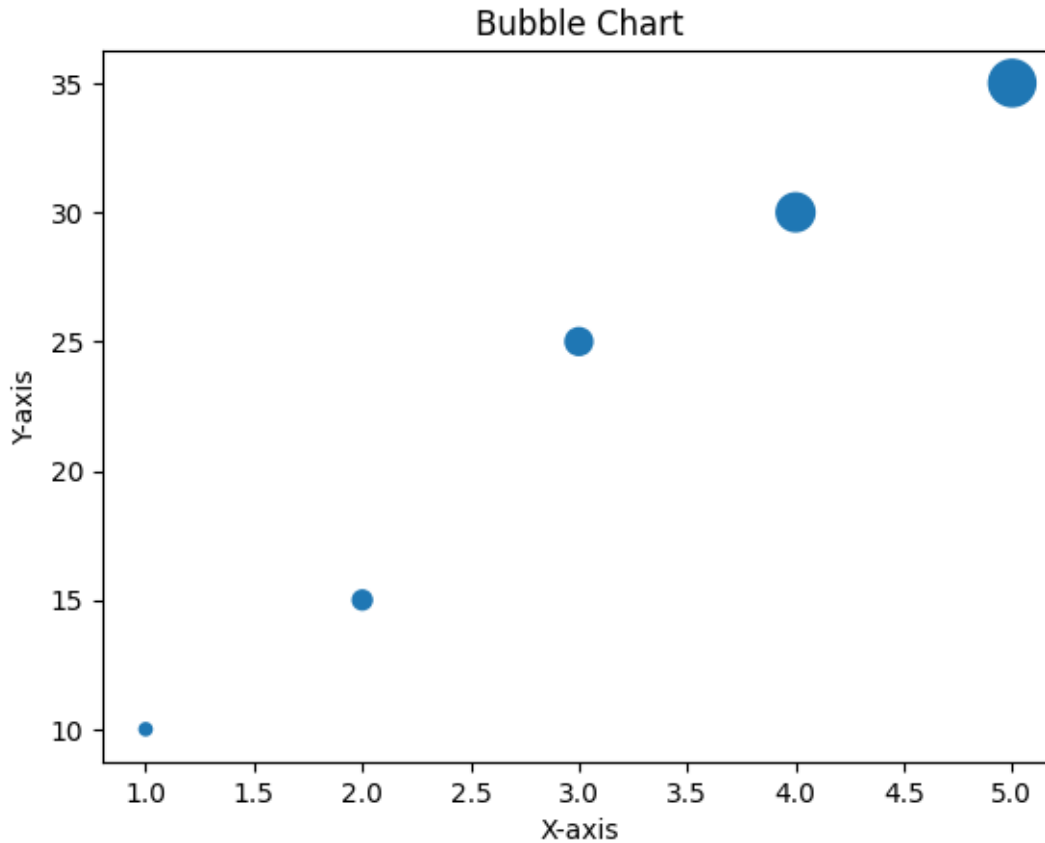


1.16 10. Bubble Chart:

```
[24]: import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 15, 25, 30, 35]
sizes = [20, 50, 100, 200, 300]

plt.scatter(x, y, s=sizes)
plt.title('Bubble Chart')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```



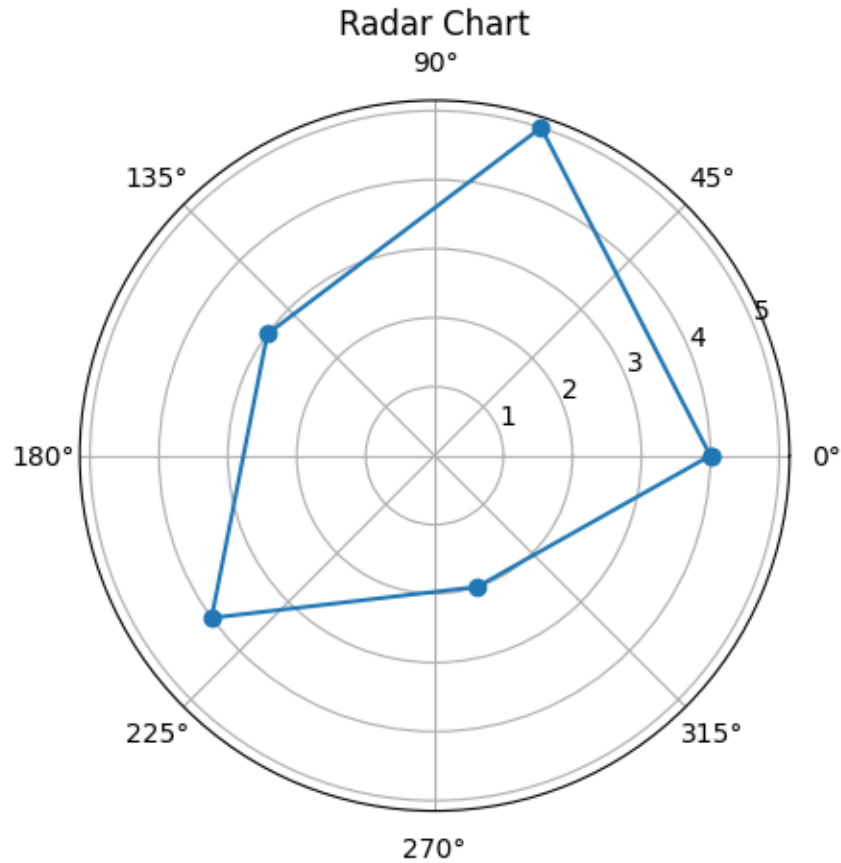
1.17 11. Radar Chart:

```
[25]: import matplotlib.pyplot as plt
import numpy as np

labels = ['A', 'B', 'C', 'D', 'E']
data = np.array([4, 5, 3, 4, 2])

angles = np.linspace(0, 2*np.pi, len(labels), endpoint=False)
data = np.concatenate((data, [data[0]])) # Close the loop
angles = np.concatenate((angles, [angles[0]]))

plt.polar(angles, data, marker='o')
plt.title('Radar Chart')
# plt.xticks(angles[:-1], labels)
plt.show()
```



1.18 12. Pie Chart:

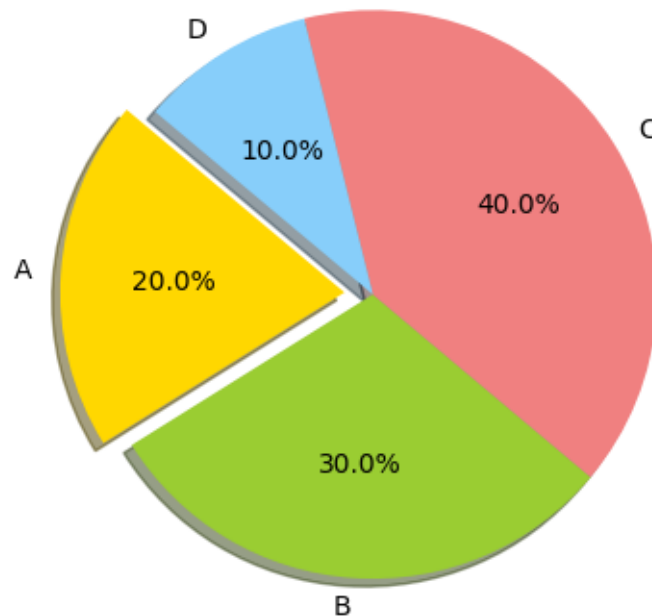
```
[26]: import matplotlib.pyplot as plt

labels = ['A', 'B', 'C', 'D']
sizes = [20, 30, 40, 10]
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
explode = (0.1, 0, 0, 0) # explode 1st slice (i.e., 'A')

plt.pie(sizes, explode=explode, labels=labels, colors=colors, autopct='%1.1f%%', shadow=True, startangle=140)
```

```
[26]: ([<matplotlib.patches.Wedge at 0x2b291f1fed0>,
<matplotlib.patches.Wedge at 0x2b291f782d0>,
<matplotlib.patches.Wedge at 0x2b291f787d0>,
<matplotlib.patches.Wedge at 0x2b291f78cd0>],
[Text(-1.1970768608146378, 0.08370776130188, 'A'),
Text(-0.07673219737211265, -1.0973204499536346, 'B'),
Text(0.9328527021418345, 0.5829115165328939, 'C'),
```

```
Text(-0.5829112236950298, 0.9328528851273191, 'D')],
[Text(-0.6982948354752053, 0.048829527426096656, '20.0%'),
Text(-0.04185392583933417, -0.598538427247437, '30.0%'),
Text(0.5088287466228187, 0.3179517362906693, '40.0%'),
Text(-0.31795157656092526, 0.5088288464330831, '10.0%')]]
```



1.19 13. Interactive Plot:

Use Case: Interactive plots that allow users to explore data interactively for dashboards and complex analysis.

Library: Plotly / Bokeh

```
[1]: import plotly.express as px
import bokeh.plotting as bp
from bokeh.models import HoverTool

# Create a new plot
p = bp.figure(plot_width=700, plot_height=400, title="Interactive Plot",
tools="pan,wheel_zoom,box_zoom,reset")
# Add a scatter plot to the figure
p.scatter([1, 2, 3, 4, 5], [10, 15, 25, 30, 35])
# Show the plot
bp.show(p)
```

```

-----
AttributeError                                Traceback (most recent call last)
Cell In[1], line 6
      3 from bokeh.models import HoverTool
      5 # Create a new plot
----> 6 p =
      ↪ bp.figure(plot_width=700, plot_height=400, title=
      7 # Add a scatter plot to the figure
      8 p.scatter([1, 2, 3, 4, 5], [10, 15, 25, 30, 35])

File d:\Private\GitHub\pythonCourse\2025-2026\
  ↪ venv\Lib\site-packages\bokeh\plotting\_figure.py:197, in figure.__init__(self
  ↪ *arg, **kw)
      195 for name in kw.keys():
      196     if name not in names:
--> 197         ↪
      ↪ self._raise_attribute_error_with_matches(name, names | opts.properties())
      199 super().__init__(*arg, **kw)
      201 self.x_range = get_range(opts.x_range)

File d:\Private\GitHub\pythonCourse\2025-2026\
  ↪ venv\Lib\site-packages\bokeh\core\has_props.py:378, in HasProps.
  ↪ _raise_attribute_error_with_matches(self, name, properties)
      375 if not matches:
      376     matches, text = sorted(properties), "possible"
--> 378 raise AttributeError(f"unexpected attribute {name!r} to {self.__class__
  ↪ __name__}, {text} attributes are {nice_join(matches)}")

AttributeError: unexpected attribute 'plot_width' to figure, similar attributes
  ↪ are outer_width, width or min_width

```

```

[ ]: import plotly.express as px # Import Plotly Express for simpler interactive
  ↪ plotting
# Sample Data
df = px.data.iris() # Load built-in Iris dataset
# Create Interactive Scatter Plot
fig = px.scatter(df, x='sepal_width', y="sepal_length", color='species',
  ↪ title="Interactive Scatter Plot Example") # Create an interactive scatter
  ↪ plot
fig.show() # Display the interactive plot

```

1.20 High-Level Interactive Visualization

```
[ ]: import holoviews as hv
import pandas as pd
hv.extension('bokeh')
df = pd.DataFrame({
    "Hours Studied": [1, 2, 3, 4, 5, 6],
    "Score": [45, 50, 60, 70, 78, 88]
})
hv.Scatter(df, kdims="Hours Studied", vdims="Score").opts(
    size=5,
    color="Score",
    cmap="Viridis",
    title="Study Hours vs Exam Score"
)
```

```
[ ]: :Scatter [Hours Studied] (Score)
```

1.21 Timeline Graphics

Timeline graphics in Python can be created using several libraries, with Plotly and Matplotlib being the most common choices. Plotly offers interactive, web-ready visualizations, while Matplotlib is a powerful tool for static plots, including specialized timeline examples in its gallery.

1.21.1 1. Using plotly (interactive Timeline/Gantt Chart):

```
[36]: import plotly.express as px
import pandas as pd
import datetime as dt

df = pd.DataFrame([
    dict(Task="Project Planning", Start='2025-01-10', Finish='2025-02-15'),
    dict(Task="Development", Start='2025-02-15', Finish='2025-03-01'),
    dict(Task="Testing", Start='2025-03-01', Finish='2025-03-05'),
    dict(Task="Launch", Start='2025-03-05', Finish='2025-03-10')
])

df['Start'] = pd.to_datetime(df['Start'])
df['Finish'] = pd.to_datetime(df['Finish'])

fig = px.timeline(df, x_start="Start", x_end="Finish", y="Task", color="Task")
fig.update_yaxes(autorange="reversed") # optional: tasks are listed from top to bottom
fig.show()
```

1.21.2 2. Using Matplotlib (static Timeline):

```
[40]: # 2. Using Matplotlib (static Timeline):
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import pandas as pd
import datetime as dt
# Sample data
data = {
    "Task": ["Project Planning", "Development", "Testing", "Launch"],
    "Start": [dt.datetime(2025, 1, 10), dt.datetime(2025, 2, 15), dt.
↳ datetime(2025, 3, 1), dt.datetime(2025, 3, 5)],
    "Finish": [dt.datetime(2025, 2, 15), dt.datetime(2025, 3, 1), dt.
↳ datetime(2025, 3, 5), dt.datetime(2025, 3, 10)]
}
df = pd.DataFrame(data)
# Plot
fig, ax = plt.subplots(figsize=(10, 6))
ax.barh(df["Task"], df["Finish"] - df["Start"], left=df["Start"])
ax.xaxis_date()
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m-%d'))
ax.set_xlabel('Date')
ax.set_ylabel('Task')
ax.set_title('Timeline')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

